

Real-Time Post-Quantum Secured Image Pipeline on NVIDIA BlueField-3 DPU

Jerónimo S. García^{1,2}, Joseph G. Meyer^{1,3}, Idelfonso Tafur Monroy³,
J. J. Vegas Olmos², and Sokol Kosta¹

¹ Department of Electronic Systems, Aalborg University, Copenhagen, Denmark
`{jsg,sok}@es.aau.dk`

² NVIDIA Corporation, Yokneam Illit, Israel
`{jsanchezgarc,jomeyer,juanj}@nvidia.com`

³ Eindhoven University of Technology (TU/e), Eindhoven, The Netherlands
`{j.g.meyer,i.tafur.monroy}@tue.nl`

Abstract. Edge AI image pipelines must decode, preprocess, and validate every frame prior to inference. With the standardization of post-quantum cryptography (PQC) to protect against future quantum threats, these pipelines must authenticate and decrypt frames as well. Conventional architectures execute these tasks either on the host CPU, incurring overheads, or on the GPU, diverting streaming multiprocessor (SM) capacity away from inference. Furthermore, both approaches expose decrypted frames within the inference host’s trust domain. We propose offloading this secure preprocessing pipeline to the Data Processing Unit (DPU). By terminating frames at network ingress, the DPU’s general-purpose cores authenticate, decrypt, and preprocess the data. The resulting inference-ready tensor is then transferred to the GPU. This approach preserves GPU SMs for inference and isolates key material and plaintext from the host. We implement and evaluate this design on an NVIDIA BlueField-3 DPU. On a single Arm core, the secure receive path sustains 1080p resolution at 85 fps for JPEG and 98 fps for TIFF. For a raw 1080p payload, the per-frame PQC stage consumes only 23% of the budget for 30 fps, comfortably meeting real-time constraints. By placing secure preprocessing on the DPU, we remove it from the inference critical path and eliminate host CPU and GPU resource contention for high-definition streams.

Keywords: Data Processing Unit, post-quantum cryptography, edge computing, image processing, BlueField-3, ML-KEM, ML-DSA

1 Introduction

Edge AI inference pipelines execute machine-learning models on image or video streams near the data source, avoiding cloud offloading to meet strict latency and bandwidth constraints [25, 26]. However, before inference can occur, each frame must be decoded, processed, validated, and transformed into a tensor. This preprocessing is computationally expensive, often accounting for 50–80%

of the total pipeline workload [6, 16]. In real-time deployments targeting 30 fps [4], decoding and preprocessing stages must not overtake this pace to avoid delaying the entire pipeline.

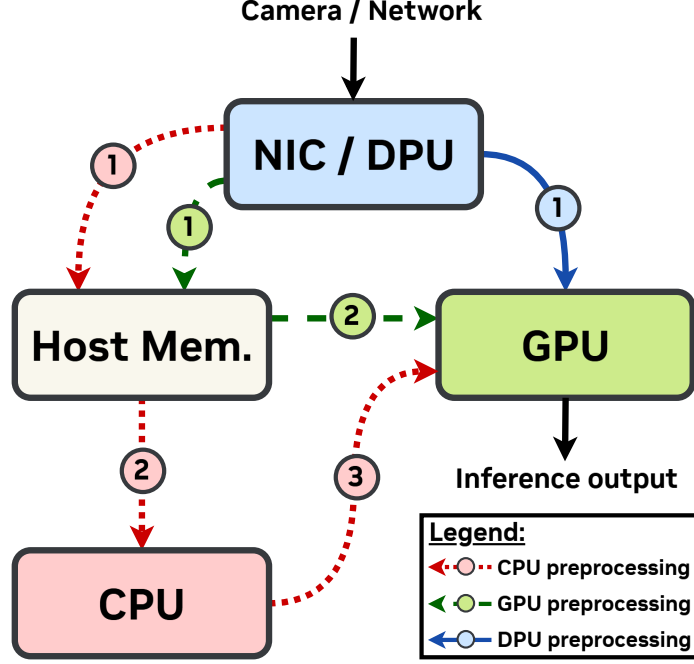


Fig. 1: Two conventional architectures for edge image pipelines. The host-CPU path stages every frame in host RAM and crosses PCIe twice (NIC→host RAM→GPU memory), spending host-CPU cycles on preprocessing. The GPU-side path eliminates one PCIe crossing (NIC→GPU memory) but runs preprocessing on the same streaming multiprocessors required by downstream inference kernels. Both motivate the DPU-resident pipeline proposed in this paper.

Conventional architectures handle preprocessing along one of two paths (Figure 1). In the *host-CPU path*, the CPU performs all preprocessing before handing frames to the GPU. While simple, every image traverses PCIe twice and is copied through host memory (NIC → host RAM, then host RAM → GPU). Consequently, the CPU constrains the accelerator it feeds, often leaving it data-starved [17, 29, 33, 34]. Alternatively, the *GPU-side path* moves preprocessing onto the GPU using data-loading frameworks (e.g., NVIDIA DALI) [15] or SM-resident libraries (e.g., NVIDIA nvImageCodec). This removes the CPU bottleneck and reduces the data movement to a single or two PCIe crossing (NIC → GPU). However, it partitions GPU compute between two mismatched workloads: inference is an embarrassingly parallel, data-regular operation that

saturates SMs [2], whereas entropy decoding and metadata parsing are serial, branch-heavy routines whose data dependencies stall execution and waste SM capacity [31]. Although fixed-function blocks, such as NVJPG [19], sidestep the SM cost for supported codecs, each GPU exposes only a small pool of such engines. High-fan-in ingestion can saturate them, stacking queueing latency on top of per-image processing time.

The security landscape introduces a second axis of pressure. In 2024, NIST finalized two post-quantum cryptographic (PQC) standards: ML-KEM (FIPS 203) for key establishment and ML-DSA (FIPS 204) for digital signatures [12, 13]. Edge deployments ingesting images over untrusted networks must increasingly authenticate and encrypt streams to ensure data captured today cannot be decrypted by future quantum computers [11]. Integrating PQC into the CPU or GPU paths forces another placement decision: software ML-KEM/ML-DSA on the host CPU is fast but keeps key handling, authentication, and plaintext staging inside the inference-host trust domain. Conversely, offloading PQC to the GPU consumes the same SMs required for inference.

We argue that Data Processing Units (DPUs), programmable network cards combining multi-core CPUs, hardware accelerators, and high-speed interconnects on a single SoC, offer a path that resolves both pressures simultaneously. Incoming frames terminate on the DPU before reaching the GPU. The DPU’s on-board cores authenticate, decrypt, decode, and preprocess each frame at the point of ingress. Only the inference-ready tensor crosses PCIe as a single DPU-to-GPU DMA. Redundant memory copies are eliminated, and the GPU’s SMs remain exclusively available for inference. This placement also redraws the security boundary: receiver-side PQC key material, authentication decisions, and plaintext staging stay within the network-side DPU domain. The inference host and GPU receive only frames that have already passed authentication.

This frames our design goal and exposes two questions: what does a PQC image pipeline for Edge AI look like, and is the DPU fast enough to absorb the full secure-pipeline path within the real-time frame budget?

Contributions. This paper makes three contributions:

1. **A DPU-resident architecture** for PQC-secured image preprocessing that terminates, authenticates, and decrypts frames at network ingress. It forwards only inference-ready tensors to the GPU, providing explicit security-domain separation: receiver-side keys, AEAD session state, plaintext staging, and authentication checks never leave the network.
2. **A characterization of secure preprocessing on BF3 Arm cores**, covering image decode and preprocessing (JPEG, PNG, TIFF, EXIF, ICC) at three resolutions together with per-frame cryptographic overhead (ML-DSA-65, SHA-256, AES-256-GCM). We identify which configurations satisfy a 30 fps budget and show that the post-quantum lattice math contributes less than 2% of the PQC cost, with the two bulk-data passes (SHA-256 signature pre-hash and AES-256-GCM) dominating the authenticated-ingress overhead.

3. **An end-to-end feasibility result for receiver-side deployment:** on a single BF3 Arm core at 1080p, the full receiver pipeline (PQC decrypt $\rightarrow$ decode $\rightarrow$ PCIe handoff) completes in 11.8 ms for JPEG and 10.2 ms for TIFF. These results show that DPU-resident secure preprocessing is practical and does not bottleneck the pipeline up to 1080p, leaving some room for further processing.

Section 2 surveys network offloading, GPU image-processing limitations, and PQC at the edge. Section 3 describes the pipeline and PQC protocol. Section 4 details the setup, ?? presents the measurements, Section 5 discusses implications and limitations, and Section 6 concludes.

2 Background and Related Work

2.1 Network Offloading

Network offloading shifts packet processing from the host CPU to the NIC using programmable SmartNICs and DPUs. These devices combine general-purpose cores, accelerators, and high-speed interfaces on a single SoC. Instead of terminating packets in host memory, DPUs parse, filter, and transform traffic as it traverses the NIC, often leveraging zero-copy buffer access and hardware support for cryptography, tunneling, and transport offload. This paradigm has been applied to infrastructure tasks [3], RDMA and storage datapaths [8], and in-network aggregation and key-value services [24]. We extend this paradigm to image preprocessing and per-frame post-quantum cryptography on the DPU’s general-purpose cores.

2.2 GPU Image Pipelines and their Limitations

GPUs excel at dense, data-parallel stages of image pipelines such as filtering, re-sizing, normalization, and matrix-based transforms, where many threads can apply the same operation independently across pixels or tiles [2]. However, end-to-end image pipelines are heterogeneous. In addition to these regular stages, they include bitstream parsing, entropy decoding, metadata extraction, colorspace interpretation, and format-specific control flow, all of which tend to expose irregular memory accesses, fine-grained dependencies, and small working sets. As a result, the benefit of moving an image pipeline to the GPU depends not only on pure throughput, but also on how each stage maps to the underlying hardware.

Codec-dependent decode acceleration. GPU-side image decoding is codec-dependent rather than uniform across formats. Table 1 summarizes current nvImageCodec [14] decode support by codec.

Some codecs, such as compressed JPEG, can use dedicated fixed-function decode hardware on supported platforms, avoiding streaming multiprocessor (SM) usage. Others, such as uncompressed JPEG and some TIFF variants, are decoded through CUDA kernels on the SMs and therefore compete with other GPU compute workloads. Formats without GPU decode support, such as PNG,

Table 1: Codec-dependent decode paths in GPU-oriented image pipelines [14].

Legend: ✓ HW decode, ◼ SM decode, ✗ CPU decode.		
Codec	Decode path	Support Scope
JPEG (compressed)	✓	Fixed-function decode; limited HW engine count
JPEG (uncompressed)	◼	Uses SMs; may contend with GPU compute
TIFF	◼	Only uncompressed or supported codecs; uses SMs
PNG	✗	No GPU decode support
BMP	✗	No GPU decode support
WebP	✗	No GPU decode support

BMP, and WebP, remain on the CPU. In practice, this means that “GPU decoding” is not a single execution model, but a mix of fixed-function, SM-based, and host-side paths with different implications for throughput and interference with downstream inference.

Offload and data-movement overheads. Even when a stage can run on the GPU, acceleration is not free. Kernel launch, synchronization, and host-device transfers introduce largely fixed costs that must be amortized over enough work. For small images or lightweight tasks such as metadata parsing and header inspection, these overheads can dominate execution time.

2.3 Post-Quantum Cryptography at the Edge

NIST standardized ML-KEM (FIPS 203 [13]) and ML-DSA (FIPS 204 [12]) in 2024, marking the transition from experimental to deployment-ready PQC. The threat that future quantum computers pose to classical public-key cryptography motivates migration well before such machines exist, since data captured today may be decrypted later [1, 11]. The effort remains active: in March 2025, NIST selected the code-based KEM HQC as a backup to ML-KEM, adding algorithmic diversity against a potential break of lattice assumptions [?]. This does not alter our per-frame design, as our architecture treats the KEM as a pluggable component (Section 3.3), but it reinforces the value of algorithm agility, which DPU-side termination provides without involving the inference host.

Existing PQC performance studies generally fall into two camps: those targeting constrained devices (microcontrollers, FPGAs, IoT nodes) to fit PQC into tight memory and energy budgets, and those evaluating PQC in network protocols, chiefly the impact of ML-KEM/ML-DSA on TLS 1.3 handshakes on server-class CPUs [21]. Recent characterizations of NIST PQC on the BlueField-3 DPU [9, 10] establish the platform’s acceleration landscape (AES-256 and SHA-2 via ARMv8 crypto extensions; SHA-3/SHAKE and asymmetric primitives in software via liboqs), but does not couple PQC with image processing or per-frame budgets. We study a different point: PQC running *alongside* image decoding on NIC-side compute under a per-frame budget, where cryptographic

work must complete within the inter-frame interval while sharing cycles with decode.

3 System Architecture

This section describes the receiver-side architecture: a pipeline that decrypts, authenticates, decodes, and preprocesses frames entirely on DPU cores at the network edge before forwarding inference-ready tensors to the GPU. We implement it on an NVIDIA BlueField-3 (BF3) DPU, but the design applies to any DPU exposing general-purpose cores, local DRAM, and a DMA engine on the host PCIe fabric. We treat the sender as a black box: any source delivering PQC-protected compressed frames over the network is compatible, regardless of the specific AEAD scheme, signature algorithm, or codec.

3.1 Pipeline Overview

The system runs on a host equipped with a DPU, a GPU, and a host CPU. The DPU owns the NIC that receives frames and co-locates its cores with local memory on the same SoC, allowing frames to be processed without involving the host CPU or crossing PCIe. Figure 2 shows the complete receiver-side data flow.

In our implementation, each frame is protected with ML-KEM-768 for key encapsulation, ML-DSA-65 for the digital signature, and AES-256-GCM for authenticated encryption. ML-KEM and ML-DSA are the post-quantum primitives standardized in FIPS 203 and FIPS 204; AES-256-GCM is the standard NIST AEAD (FIPS 197 with SP 800-38D), retained as a symmetric construction whose 256-bit key remains classically and quantum-resistant under known attacks. Any compatible combination of a PQC KEM, a PQC signature scheme, and an AEAD cipher can be substituted without architectural changes.

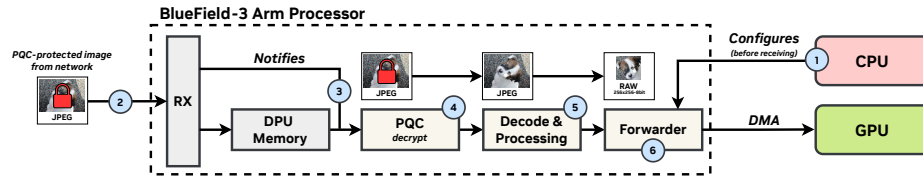


Fig. 2: End-to-end receiver pipeline on the BF3 SoC. A PQC-protected JPEG frame arrives, is decrypted and decoded inside the DPU, and emerges as a RAW 256×256 tensor ready for inference. The numbered circles ①–⑥ mark the CPU configuration, network ingress, PQC decryption, image decode and processing, and final DMA transfer stages corresponding to Section 3.1.

The pipeline proceeds in numbered stages (Figure 2): ① Before any frame arrives, the host CPU configures the DPU forwarder with the destination, a GPU

VRAM region or host buffer for the final tensor. Control-plane events (session establishment, forwarder configuration, error recovery) run on the host CPU; once configured, it does not participate in the per-frame data path. ② The NIC receives the compressed, PQC-protected frame and writes it directly into DPU-local memory. ③ The NIC notifies the DPU processor. ④ The sealed frame is opened: AES-256-GCM authenticated decryption recovers the compressed payload and validates the AEAD tag. The DPU then recomputes the SHA-256 digest and verifies the ML-DSA-65 signature. A failure at any check drops the frame silently, logs the event, and continues with the next. ⑤ The authenticated payload is decoded into a raw pixel buffer and subjected to application-specific preprocessing (resize, color conversion, normalization). ⑥ The forwarder DMAs the preprocessed tensor directly to the pre-configured destination. The frame crosses PCIe exactly once, arriving inference-ready with no further copies.

This sidesteps the dominant costs of conventional paths. *Data crosses PCIe exactly once*: only the finished tensor is forwarded, eliminating the second crossing incurred by the host-CPU path. *GPU SMs are reserved exclusively for inference*: preprocessing and PQC run on dedicated DPU cores and never compete with inference kernels. As a secondary benefit, PQC support is native to the DPU, keeping the host and GPU unaware of the cryptographic layer.

3.2 Security-Domain Separation

The DPU placement serves as a security boundary, not just a latency optimization. In a conventional path, sealed frames arrive at the NIC, are DMAed into host memory, and are opened by host CPU or GPU. Consequently, PQC keys, signature-verification state, AEAD session keys, plaintext staging buffers, and authentication-failure handling all reside within the inference-host domain before a frame is accepted.

In the DPU path, sealed frames terminate on the network at ingress. The DPU cores perform AES-256-GCM opening, SHA-256 hashing, and ML-DSA-65 verification in DPU-local memory. Failed frames are dropped before exposure to host or GPU memory. Only authenticated plaintext crosses PCIe. The DPU changes the trust boundary by isolating receiver-side key material, session keys, and unauthenticated plaintext from the inference host.

Threat model and scope. We assume an adversary can observe, modify, drop, or replay frames before they reach the receiver, and that malformed or unauthenticated frames must not be exposed to host or GPU memory. The DPU is therefore part of the cryptographic trusted computing base for ingress, while the host CPU and GPU are excluded from the decryption path. This is an architectural separation, not a formal trusted-execution claim: it does not prove side-channel resistance, protect against a fully compromised DPU, or prevent a compromised host from misusing frames after authenticated forwarding.

3.3 PQC Protocol Design

The protocol operates in two phases: a one-time session establishment that authenticates endpoints and derives symmetric keys, and a per-frame phase that seals and opens each compressed image.

Session establishment. The receiver generates an ML-KEM-768 [13] key-pair and publishes its public key. The sender encapsulates a shared secret against it, and both parties derive the AES-256-GCM session key from the result. The endpoints also exchange ML-DSA-65 public keys to support bidirectional authentication, though in the unidirectional streaming case benchmarked here, only the sender’s signature key is used per frame. Setup cost is amortized over all subsequent frames and is negligible at pipeline scale.

Per-frame sealing (sender). For each compressed image, the sender (1) computes a SHA-256 digest over the payload, (2) signs the digest with ML-DSA-65, and (3) encrypts the concatenation of payload and signature with AES-256-GCM, using the fixed-size wire header as additional authenticated data (AAD). Because the signature is encapsulated inside the ciphertext, it is never visible to an on-path observer.

Per-frame opening (receiver). Each sealed frame passes through the inverse pipeline on the DPU: AES-256-GCM decryption with the wire header as AAD (detecting any header modification), SHA-256 over the recovered payload, and ML-DSA-65 verification of the signature on that digest. Any failure in AEAD authentication, hash check, or signature verification drops the frame before forwarding.

4 Experimental Setup

4.1 Hardware Platform

The testbed comprises two NVIDIA BlueField-3 DPUs [18], each with 16 Arm Cortex-A78 cores at 2.0 GHz and 32 GB of DDR5, connected over a 100 Gbps link and running DOCA 2.7 on Ubuntu 22.04.4 LTS (aarch64). Table 2 summarizes the configuration. The downstream inference target is an NVIDIA H100 80GB HBM3 GPU. The GPU acts as the consumer of the pipeline output, not a comparison platform in this work.

Table 2: BlueField-3 DPU platform.

Parameter DPU (BF3 P-Series)	
Processor	16× Arm Cortex-A78 @ 2 GHz
Memory	32 GB DDR5
Power	≤75 W (board)
Network	100 Gbps
OS / Arch	Ubuntu 22.04.4 LTS (aarch64), DOCA 2.7

4.2 Benchmark Methodology

Image decoding. A custom C harness exercises five libraries: libjpeg-turbo [7] (JPEG), libpng [22] (PNG), libtiff [30] (TIFF), libexif [32] (EXIF), and lcms2 [23] (ICC). Three resolutions are tested (256×256 , 1920×1080 , and 3840×2160), each using a synthetic random-noise image with a fixed seed. Every configuration runs 1 000 timed iterations across three trials, preceded by 100 discarded warmup iterations. Execution is single-threaded with the benchmark thread pinned via `taskset` and the CPU governor locked to `performance`. Source images are preloaded and output buffers pre-allocated and reused, eliminating filesystem and allocator noise.

Post-quantum cryptography. A separate C harness benchmarks PQC primitives from liboqs [28] (ML-KEM-768, ML-DSA-65) and OpenSSL [20] (AES-256-GCM, SHA-256) on the DPU Arm cores, under the same protocol. We measure each primitive individually and define the composite receive operation `open_frame` = decrypt + hash + verify.

4.3 End-to-End Pipeline

The integration test connects DPU₁ (sender) to DPU₂ (receiver) over TCP. DPU₁ seals each frame and transmits the ciphertext; DPU₂ executes the full receiver-side pipeline: `open_frame`, codec decode, and DMA of the inference-ready buffer to host-mapped GPU memory. We measure 100 frames per configuration and report per-stage latencies (Figure 5).

4.4 Image Decode and Preprocessing Latency

Figure 3 reports per-operation tail latency (p99) on the BF3 Arm cores across the three resolutions, with the dashed line marking the 33.3 ms budget for 30 fps.

All pixel-touching operations scale approximately linearly with pixel count: the $\approx 127\times$ growth between 256×256 and 4K matches the pixel-count ratio. TIFF decode is cheapest at every resolution (8.2 ms at 4K) and clears the budget at all sizes, since uncompressed I/O is memory-bound and performs essentially no per-pixel computation. JPEG decode processes the image in fixed 8×8 blocks with bounded per-block cost (IDCT + Huffman), reaching 10.8 ms at 1080p and 43.2 ms at 4K. Thus, a single Arm core sustains real-time JPEG decode at 1080p but falls short at 4K. PNG decode is the most expensive codec path, roughly $4\times$ the per-pixel cost of JPEG (43.1 ms vs 10.8 ms at 1080p), and exceeds the budget at 1080p because DEFLATE’s inflation step (LZ77 + Huffman) is inherently sequential [27, 31].

EXIF metadata parsing completes in 29 μ s regardless of resolution, as the parser reads a fixed-size header without touching pixel data. ICC color-space conversion (sRGB $\rightarrow$ linear, via lcms2) is bandwidth-bound and scales linearly with the 3-channel $H \times W$ buffer, reaching 76.2 ms at 4K, more than twice the real-time budget.

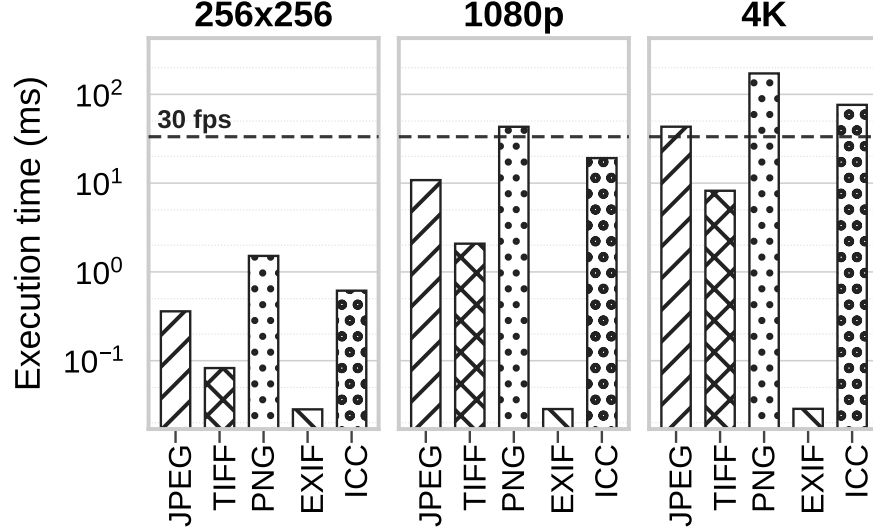


Fig. 3: Per-image execution time on the BF3 (log scale) for image codec decode and metadata/color operations at three resolutions, with JPEG q75, PNG zlib c6, and uncompressed TIFF. The dashed line marks the 33.3 ms budget for 30 fps. Bars report the tail latency (p99).

4.5 PQC Decrypt and Verify Overhead

Figure 4 decomposes the per-frame cost of the DPU receive path, ML-DSA-65 verify, SHA-256 integrity check, and AES-256-GCM decrypt, across the three uncompressed resolutions. All values tail latencies (p99).

The breakdown exposes the path’s scaling structure. ML-DSA-65 verification is resolution-independent (104 μ s): the signature is computed over the fixed 32-byte SHA-256 digest rather than the image payload, and the lattice math runs in software via liboqs with SHAKE-256 calls bounded by that fixed input. SHA-256 and AES-256-GCM, by contrast, scale linearly with payload size and therefore dominate the per-frame budget at any non-trivial resolution. Both are hardware-accelerated via the ARMv8 SHA2 and AES/PMULL extensions on the Cortex-A78AE cores; the resulting single-core steady-state ceilings, ≈ 1.4 GB/s for SHA-256 and ≈ 2.0 GB/s for AES-256-GCM, set the slope of those two bars and let the per-frame overhead be projected to any other resolution or frame rate. Verify therefore contributes under 2% at 1080p and 4K.

Mapped onto real-time budgets, the path is comfortably feasible at small and medium resolutions: $\approx 1\%$ of the 30 fps budget at 256×256 and $\approx 24\%$ at 1080p. At 4K, even the worst-case frame reaches 30.75 ms ($\approx 92\%$ of budget): nominally feasible, but leaving only ≈ 2.6 ms per frame for downstream decode, processing, and forwarding. Notably, the quantum-safe primitive itself contributes under 1%

of the 4K cost; the budget is consumed by the two bulk-data primitives (SHA-256 and AES-256-GCM), both already on the in-core ARMv8 crypto extensions.

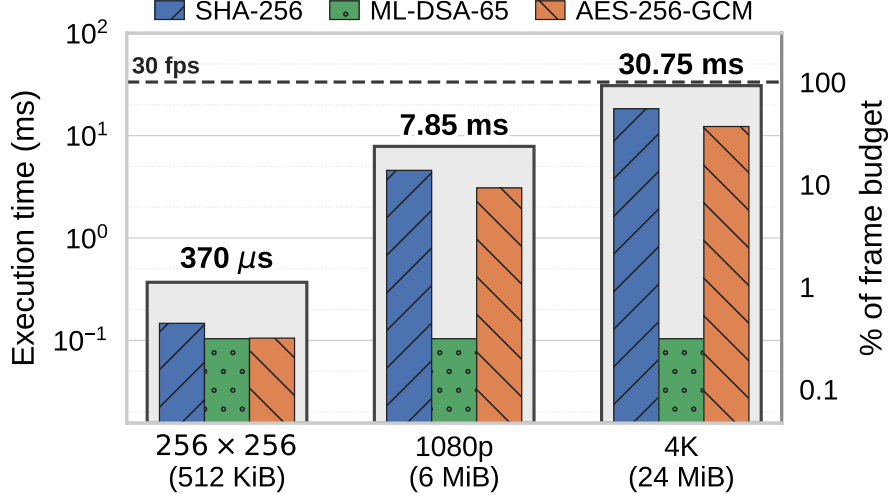


Fig. 4: DPU PQC decrypt per-frame cost (p99 latency) across raw image resolutions; outer containers give the total, inner bars the per-component breakdown.

4.6 End-to-End Pipeline

Figure 5 reports the end-to-end receiver runtime for the three codecs at 256×256 and 1080p, the resolutions that leave sufficient 30 fps budget for downstream stages. The pipeline starts when the DPU receives the PQC-protected encoded frame and ends with the decoded, uncompressed image in GPU memory. It comprises three stages: PQC decrypt (authentication and AES-GCM decrypt on the Arm cores), image decode on the DPU, and the raw host-to-device (H2D) PCIe transfer.

At 256×256 , all three codecs sit well below the budget: JPEG completes in $534 \mu s$, TIFF in $480 \mu s$, and PNG in $1.76 ms$. For natively compressed formats, the decoder dominates ($\sim 66\%$ of the JPEG pipeline, $\sim 85\%$ of PNG), while PQC decrypt ($\sim 33\%$ and $\sim 14\%$) and the PCIe transfer ($\sim 2\%$ and $<1\%$, $\sim 10 \mu s$) are comparatively cheap. TIFF is the outlier: because its “compressed” payload is already the raw RGB image, PQC decrypt acts on the full uncompressed buffer and dominates at $\sim 83\%$, while the decoder reduces to memory copies.

The same trends carry to 1080p at higher absolute cost. PNG takes $46.5 ms$, $3.9\times$ slower than JPEG ($11.8 ms$), and the PNG decoder alone ($43.1 ms$) exceeds the budget. PNG sustains only ~ 21 fps on a single core, necessitating multi-core

decomposition across the BF3’s 16 Arm cores for real-time performance. JPEG and TIFF instead retain ~ 21.6 ms and ~ 23.1 ms of headroom below the 30 fps line (theoretical ceilings of ~ 85 and ~ 98 fps), leaving budget for downstream work. For JPEG, the decoder share rises from $\sim 66\%$ at 256^2 to $\sim 92\%$ at 1080p: even though both decoder cost and compressed payload scale roughly linearly with pixel count ($\sim 31\times$), PQC has a fixed per-frame setup (verify + framing) that becomes negligible once the payload is large enough for the throughput term to dominate. Thus, PQC decrypt grows only $\sim 3.8\times$ and its share collapses to $\sim 6\%$. TIFF again behaves as a raw-payload outlier: PQC decrypt processes the full 6.22 MB image, taking 7.8 ms ($\sim 77\%$), and the decoder reduces to copies. Across all 1080p bars, the PCIe H2D transfer ($329\mu\text{s}$ for 6.22 MB, ~ 18.9 GB/s) accounts for at most $\sim 3\%$ of the end-to-end latency.

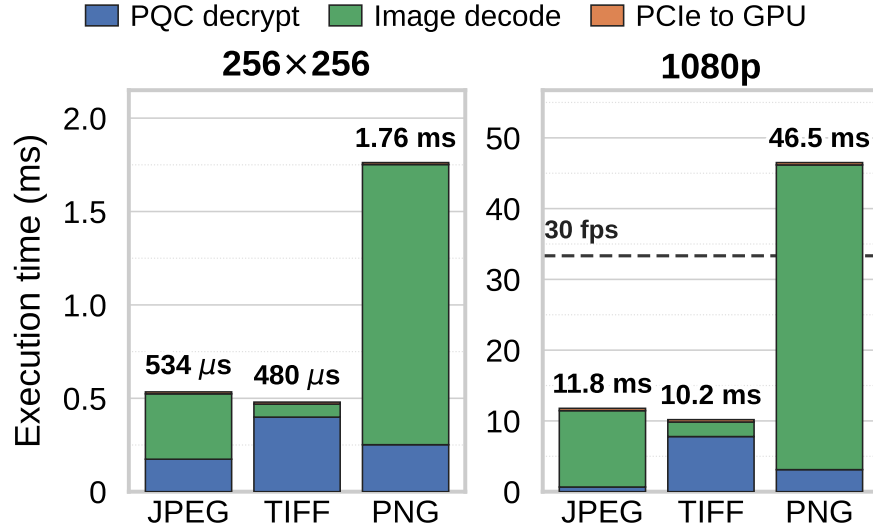


Fig. 5: Receiver-side end-to-end pipeline latency on the BF3 DPU. Stacked bars show the three stages from PQC-protected frame arrival to inference-ready tensor in GPU memory: PQC decrypt, image decode, and the PCIe H2D transfer to GPU memory. The horizontal line marks the 33.3 ms budget for 30 fps.

5 Discussion

5.1 Codec-Dependent Feasibility

The measurements suggest a codec-dependent feasibility picture. TIFF and JPEG comfortably meet 30 fps at 1080p on one Arm core, with substantial headroom

for downstream preprocessing, and EXIF is effectively free at 26 μ s constant time. PNG is the only codec that misses 1080p real-time on a single core due to DEFLATE inflation’s sequential nature. However, distributing it across the BF3’s 16 cores brings real-time within reach. Crucially, there is no production-grade GPU PNG decoder to fall back on, making the DPU the only placement that avoids an extra PCIe round-trip for this format. ICC color conversion is the most expensive per-pixel operation and exceeds the budget at 4K on a single core, again pointing to multi-core decomposition. For JPEG specifically, a GPU’s fixed-function NVJPG block decodes faster per image than an Arm core [5, 19]; the DPU’s value there is system-level, not per-image: it decouples ingress from the GPU’s small fixed pool of decode engines under high fan-in. When it also performs the resize and normalization to the inference input resolution, the tensor crossing PCIe shrinks below the JPEG payload size. On a single core, the only codec that misses 1080p real time is PNG; the rest leave ample headroom for downstream preprocessing.

5.2 Bulk-Data Costs in the PQC Path

Figure 4 highlights a critical point: the lattice math itself is negligible. The asymmetric, quantum-safe primitive (ML-DSA-65 verify) operates on a fixed 32-byte digest and contributes under 2 % of the per-frame PQC cost at 1080p and under 1 % at 4K. The per-frame budget is instead consumed by two bulk-data passes, SHA-256 and AES-256-GCM, both hardware-accelerated through ARMv8 extensions. It is important to attribute these two passes correctly. AES-256-GCM is the AEAD layer: it provides confidentiality and symmetric integrity, and any encrypted-ingress pipeline, post-quantum or classical, pays this cost (3.07 ms at 1080p). The SHA-256 pass, by contrast, is *not* generic AEAD overhead: it is the pre-hash that feeds the ML-DSA-65 signature (HashML-DSA), and a pipeline without origin authentication would not incur it.

Consequently, the marginal cost of *adding* post-quantum origin authentication on top of an AEAD-only pipeline is the full-payload SHA-256 pass (4.56 ms at 1080p) plus the 101 μ s verify: roughly 60 % of the measured PQC stage, not the verify alone. The encouraging result is therefore not that the signature is “free,” but that its dominant cost reduces to a single bulk hash that already runs on the in-core ARMv8 SHA2 engine at ≈ 1.4 GB/s, rather than to expensive lattice arithmetic. Where the signature’s integrity guarantee overlaps with the AEAD tag, deployments that need only symmetric integrity can omit the SHA-256 pass entirely and recover that 60 %; those that require non-repudiable, post-quantum origin authentication pay one extra bulk hash and a sub-1 % verify.

5.3 Placement Over Peak Throughput

A high-end host CPU runs these primitives faster than a single Arm core, and prior TLS-PQC studies confirm that server-class x86 absorbs ML-KEM/ML-DSA cheaply [21]. However, raw throughput and system placement answer different questions. The DPU’s value lies in terminating cryptography at ingress,

keeping key material and unauthenticated plaintext off the inference host, and forwarding only authenticated frames, all while preserving host CPU and GPU capacity for inference. What matters for performance is not whether the Arm core is the fastest cryptographic engine, but whether it meets the frame budget, which it does at 1080p for JPEG and TIFF codecs.

5.4 Scaling Constraints at 4K

The 4K case is the most informative for identifying binding constraints. At $\approx 92\%$ of the 30 fps budget, the receive path is feasible but leaves only ≈ 2.7 ms per frame for downstream decode and forwarding. This is insufficient for JPEG (43.2 ms at 4K on one core) or PNG (171.9 ms). Closing the gap requires parallelism and offload. AES-256-GCM admits multiple offload paths on current BF3 silicon: a dedicated AES-GCM block (the AES_GCM_5 / Memory-to-Memory Offload engine) and ConnectX-7 inline IPsec/TLS that removes AES from the application path entirely. SHA-256 has no equivalent dedicated engine on BF3 (the DOCA SHA library is unsupported on this generation), so SHA can be scaled only by sharding across additional Arm cores, making the SHA pass the binding constraint at 4K.

5.5 Limitations

Several aspects bound the scope of these conclusions. The codec microbenchmarks and the end-to-end pipeline run on a single pinned Arm core; the 16-core BF3 complex offers parallelism we have not characterized, and multi-stream contention on the shared L3 and DDR5 controllers is left to future work. Hardware crypto offload on the BF3 DPA and other device engines also remains future work, so the reported PQC numbers should be read as software-on-Arm measurements. The end-to-end test uses a 100 GbE link, and the balance between network, crypto, and decode may shift at higher line rates. Finally, the security claim is an architectural separation argument: it keeps keys, authentication checks, and plaintext staging off the inference host/GPU path, but it is not a formal TEE design or a proof of side-channel isolation.

6 Conclusion

We have demonstrated that a complete PQC-secured image-preprocessing path (authentication, AEAD decryption, codec decode, and PCIe handoff to the GPU) runs in real time on the Arm cores of an NVIDIA BlueField-3 DPU at 256×256 and 1080p, without host CPU or GPU involvement in the critical path. On a single Cortex-A78AE core at 1080p, the receiver-side pipeline completes in 11.8 ms for JPEG and 10.2 ms for TIFF, sustaining 85 and 98 fps against a 30 fps target. For a raw 1080p payload, the per-frame PQC stage accounts for 23 % of the budget, of which the post-quantum signature verify is under 2 %; the

remainder is bulk hashing and decryption already accelerated by the ARMv8 crypto extensions.

What makes the design compelling is where it executes the work, not simply how fast it runs each primitive. Authenticating, decrypting, and decoding every frame on the DPU before it crosses PCIe removes the host round-trips conventional pipelines incur. It also keeps key material and plaintext off the inference host and GPU, preserving SMs for inference. Because a single Arm core meets the 1080p budget for common codecs, this isolation costs no inference throughput. Future work will characterize multi-stream scaling across the 16-core Arm complex and evaluate offloading bulk-data crypto (AES-256-GCM in particular) to the BF3 AES_GCM_5 / MMO engine, establishing DPU-resident PQC-secured preprocessing as a practical default for real-time 1080p edge deployments where security-domain separation and GPU-SM preservation are paramount.

Acknowledgments. This work was partly funded by the QUARC project of the European Union (EU) Horizon Europe research and innovation program within the framework of Marie Skłodowska-Curie Actions under grant number 101073355, as well as the EDGEAI “Edge AI Technologies for Optimised Performance Embedded Processing” project under grant number [TODO: grant number].

Disclosure of Interests. Generative AI tools were used to assist with literature review and editing of this manuscript; all technical content was verified by the authors.

References

1. Bernstein, D.J., Lange, T.: Post-quantum cryptography. In: *Nature*. vol. 549, pp. 188–194. Springer (2017). <https://doi.org/10.1038/nature23461>
2. Dongarra, J., Abalenkovs, M., Abdelfattah, A., Gates, M., Haidar, A., Kurzak, J., Luszczek, P., Tomov, S., Yamazaki, I., YarKhan, A.: Parallel Programming Models for Dense Linear Algebra on Heterogeneous Systems. *Supercomput. Front. Innov.: Int. J.* **2**(4), 67–86 (Mar 2015). <https://doi.org/10.14529/jsfi150405>, <https://doi.org/10.14529/jsfi150405>
3. Firestone, D., Putnam, A., Mundkur, S., Chiou, D., Dabagh, A., Andrewartha, M., Angepat, H., Bhatt, V., Caulfield, A., Chung, E., et al.: Azure accelerated networking: SmartNICs in the public cloud. In: *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. pp. 51–66 (2018)
4. Guo, C., Wu, H., Deng, Z., Soni, G., Ye, J., Padhye, J., Lipshteyn, M.: Real-time video processing at the edge. In: *Proceedings of ACM SIGCOMM* (2019)
5. Iglovikov, V.: Benchmarking JPEG decoders on CPU and GPU (2025)
6. Kang, D., Mathur, A., Veeramacheneni, T., Bailis, P., Zaharia, M.: Jointly optimizing preprocessing and inference for DNN-based visual analytics. *Proceedings of the VLDB Endowment* **14**(2), 87–100 (Oct 2020). <https://doi.org/10.14778/34258179.3425881>, <https://dl.acm.org/doi/10.14778/3425879.3425881>
7. libjpeg-turbo contributors: libjpeg-turbo (2024), <https://libjpeg-turbo.org/>, version 3.0. Accessed: 2025-01-15

8. Liu, M., Peter, S., Krishnamurthy, A., Phothilimthana, P.M.: Offloading distributed applications onto SmartNICs using iPipe. *ACM SIGCOMM Computer Communication Review* **49**(4), 318–333 (2019). <https://doi.org/10.1145/3341302.3342079>
9. Meinel, L.: Accelerating Post-Quantum Cryptography Algorithms on the Nvidia Bluefield 3 DPU: An Active Analysis (Jul 2024). <https://doi.org/10.22032/DBT.61764>, https://www.db-thueringen.de/receive/dbt_mods_00061764
10. Meyer, J., Mentovich, E., Tafur Monroy, I., Jose Vegas Olmos, J.: Offloading Post-Quantum Cryptography to Data Processing Units: A Performance Analysis of HQC Key Encapsulation. *IEEE Access* **14**, 53831–53845 (2026). <https://doi.org/10.1109/ACCESS.2026.3681219>, <https://ieeexplore.ieee.org/document/11475462>
11. Mosca, M.: Cybersecurity in an era with quantum computers: Will we be ready? *IEEE Security & Privacy* **16**(5), 38–41 (2018). <https://doi.org/10.1109/MSP.2018.3761723>
12. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard (FIPS 204). Federal Information Processing Standards Publication 204, NIST (2024), <https://csrc.nist.gov/pubs/fips/204/final>, ML-DSA
13. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203). Federal Information Processing Standards Publication 203, NIST (2024), <https://csrc.nist.gov/pubs/fips/203/final>, ML-KEM
14. NVIDIA: nvImageCodec Documentation, <https://docs.nvidia.com/cuda/nvimagecodec/>
15. NVIDIA: Rapid Data Pre-Processing with NVIDIA DALI (Oct 2021), <https://developer.nvidia.com/blog/rapid-data-pre-processing-with-nvidia-dali/>
16. NVIDIA: Increasing Throughput and Reducing Costs for AI-Based Computer Vision with CV-CUDA (May 2023), <https://developer.nvidia.com/blog/increasing-throughput-and-reducing-costs-for-computer-vision-with-cv-cuda/>
17. NVIDIA: Build High-Performance Vision AI Pipelines with NVIDIA CUDA-Accelerated VC-6 (Sep 2025), <https://developer.nvidia.com/blog/build-high-performance-vision-ai-pipelines-with-nvidia-cuda-accelerated-vc-6/>
18. NVIDIA Corporation: NVIDIA BlueField-3 DPU Data Sheet (2024), <https://www.nvidia.com/en-us/networking/products/data-processing-unit/>, accessed: 2025-01-15
19. NVIDIA Corporation: nvJPEG: GPU-Accelerated JPEG Decoding (2024), <https://developer.nvidia.com/nvjpeg>, accessed: 2025-01-15
20. OpenSSL Software Foundation: OpenSSL cryptography and SSL/TLS toolkit (2024), <https://www.openssl.org/>, accessed: 2025-01-15
21. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking post-quantum cryptography in TLS. In: *Post-Quantum Cryptography (PQCrypto)*. pp. 72–91. Springer (2020). https://doi.org/10.1007/978-3-030-44223-1_5
22. Randegger, G., et al.: libpng: The official PNG reference library (2024), <http://www.libpng.org/pub/png/libpng.html>, accessed: 2025-01-15
23. Saguer, M.M.: Little CMS: Great color at small footprint (2024), <https://littlecms.com/>, accessed: 2025-01-15
24. Sapio, A., Canini, M., Ho, C.Y., Nelson, J., Kalber, P., Krishnamurthy, A., Moshref, M., Ports, D.R.K., Richtarik, P.: Scaling distributed machine learning with in-network aggregation. In: *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (2021)
25. Satyanarayanan, M.: The emergence of edge computing. *Computer* **50**(1), 30–39 (2017). <https://doi.org/10.1109/MC.2017.9>

26. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE Internet of Things Journal* **3**(5), 637–646 (2016). <https://doi.org/10.1109/JIOT.2016.2579198>
27. Sitaridi, E., Mueller, R., Kaldewey, T., Lohman, G., Ross, K.A.: Massively-Parallel Lossless Data Decompression. In: 2016 45th International Conference on Parallel Processing (ICPP). pp. 242–247 (Aug 2016). <https://doi.org/10.1109/ICPP.2016.35>, <https://ieeexplore.ieee.org/document/7573824>, ISSN: 2332-5690
28. Stebila, D., Mosca, M.: Open quantum safe: liboqs — C library for quantum-resistant cryptographic algorithms (2024), <https://openquantumsafe.org/>, accessed: 2025-01-15
29. Tang, L., Zhang, D., Chen, J., Huang, P., Jin, F., Xu, C., Chen, Y., Sun, F., Chen, G.: MultiPath Transfer Engine: Breaking GPU and Host-Memory Bandwidth Bottlenecks in LLM Services (Dec 2025), <https://arxiv.org/abs/2512.16056v1>
30. Warmerdam, S., et al.: libtiff: TIFF library and utilities (2024), <https://libtiff.gitlab.io/libtiff/>, accessed: 2025-01-15
31. Weißenberger, A., Schmidt, B.: Accelerating JPEG Decompression on GPUs (Nov 2021), <https://arxiv.org/abs/2111.09219v1>
32. Wernicke, L., et al.: libexif: EXIF parsing library (2024), <https://libexif.github.io/>, accessed: 2025-01-15
33. Yang, C.C., Cong, G.: Accelerating Data Loading in Deep Neural Network Training. In: 2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC). pp. 235–245 (Dec 2019). <https://doi.org/10.1109/HiPC.2019.00037>, <http://arxiv.org/abs/1910.01196>, arXiv:1910.01196 [cs]
34. Zolnouri, M., Li, X., Nia, V.P.: Importance of Data Loading Pipeline in Training Deep Neural Networks (Apr 2020). <https://doi.org/10.48550/arXiv.2005.02130>, <http://arxiv.org/abs/2005.02130>, arXiv:2005.02130 [cs]